AICTE VERIFIED STUDENT REGISTER

Validation, Seat Ledger and Conflict Rules

Deterministic validation contract, severity handling, capacity invariants, enrolment lifecycle and test examples.

Document: 07-RULES Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Core processing rule

Golden rule
Uploaded rows are staging data. A batch can become official only after required validation, correction/review, institution-head certification and a safe commit. A risk flag alone must not block a record unless an independently defined deterministic rule requires that outcome.

2. Seven validation layers

Layer	What is checked	Severity / outcome	Notes for implementation
L1 Schema	Required fields, types, enums, dates, allowed value shapes.	ERROR for missing/invalid mandatory values.	Use one server-authoritative validation service; browser checks are convenience only.
L2 External identity / APAAR mock	Identifier format and mock match status; name/DOB match only where demo fixture supports it.	Invalid demo format = ERROR; mismatch = WARNING or explicit state; external service down/opt-out = PENDING.	Do not call live registry; name matching threshold and fields must be documented if used.
L3 Duplicate in batch	Same student repeated in same batch/institution/course-year according to agreed duplicate key.	ERROR on duplicate row; preserve source row references.	Do not merge records silently.
L4 Seat ledger	Capacity for course/year/bucket from approved demo configuration.	ERROR if reservation would exceed configured capacity.	Capacity must be sourced from stored configuration; no guessed real quota percentages.
L5 Cross-college	Same student has apparently active enrolments at two institutions in the same academic year.	WARNING + conflict case.	False positives are possible; transfer/lifecycle resolution and authorised review are required.
L6 Consistency	Fields and admission conditions agree with the configured rules.	ERROR or WARNING according to explicit rule definition.	Examples: course/year conflict, invalid lifecycle transition, nationality flag conflict; do not assert unverified policy.
L7 Risk signals	Batch/institution patterns such as late additions, repeated contact details, unusual ratios.	RISK_FLAG only; does not by itself block or prove wrongdoing.	Record signal reasons and ruleset version; false-positive feedback must not erase original evidence.

3. Severity definitions

Severity	Meaning	Can batch proceed?	Expected UI wording
ERROR	A deterministic required condition failed.	No certification/commit while blocking error is unresolved.	Explain field/row and how to correct it.
WARNING	Data may be valid but needs review or justification.	Only according to explicit warning-disposition policy. Never silently ignore.	Explain why it is a warning and what review is required.
PENDING	A check cannot yet complete or is in a permitted pending state.	Depends on explicit policy; default is do not imply verified.	State which check is pending and retry/alternate process.
INFO	Non-blocking informational outcome.	Yes, unless another rule blocks.	Present as contextual information.
RISK_FLAG	Pattern deserves investigation.	Does not automatically block.	Show signal, evidence and review status; no guilt language.

4. Illustrative synthetic data cases

Case	Input condition	Expected result
Missing field	courseCode empty	L1 ERROR; certification blocked until corrected.
Bad identifier format	Mock identifier contains 11 digits where the demo format requires 12	L2 format ERROR; this checks format only, not real-world identity.
Duplicate row	Same demo studentRef repeated within same batch/course/year	L3 ERROR with both row numbers.
NRI over capacity	Configured capacity is 3; four NRI seats would be reserved	L4 ERROR and no overflow commit.
Valid additional seats	Total exceeds approved intake due to explicitly configured additional buckets, and every bucket remains within configured limits	Do not trigger generic total-only overflow; accept capacity only if configuration/approval authorises those buckets.
Transfer not recorded	Student has an active record in two colleges in same year	L5 WARNING and conflict case; no automatic fraud verdict.
Registry unavailable	Mock adapter returns unavailable	PENDING/retry state, not fake verification success.
Repeated phone	Many synthetic students share a phone field	L7 signal and evidence; human review.
Correction made	Operator fixes one validation field	Revalidate relevant row; keep a record of resolution.
Rule changed after validation	Seat-policy version changes before certification	Revalidate or reject stale batch under documented version policy.

5. Seat ledger model

Suggested unique ledger identity: course/scope + academicYear + bucketCode. Store capacity, filled count, effective policy version and update timestamps. Every capacity must carry provenance: demo fixture ID or authorised approval reference. In a prototype, capacity can be deliberately fictional but must be labelled as such.

Config example - no real quota values

```
{
  "academicYear": "2026-27",
  "policyVersion": "DEMO-v1",
  "sourceType": "SYNTHETIC_DEMO_CONFIG",
  "buckets": [
    { "code": "REGULAR", "capacity": 57, "countsWithinIntake": true },
    { "code": "NRI", "capacity": 3, "countsWithinIntake": true },
    { "code": "TFW", "capacity": 3, "countsWithinIntake": false },
    { "code": "INTERNATIONAL", "capacity": 3, "countsWithinIntake": false }
  ]
}
```

These capacities are example test data only. They are not official policy and must never become production defaults. Keep the sourceType visible in the configuration and UI.

6. Commit and transaction invariants

- Certification service must re-check current batch status and blockers.
- Seat checks and seat-counter updates must be atomic with enrolment writes in the supported commit workflow.
- If any required write fails, abort/rollback the complete commit.
- Use a MongoDB replica set or sharded cluster to exercise multi-document transactions.
- Prevent duplicate certification and duplicate retries from committing a batch more than once.
- Use an idempotent state transition and explicit transaction outcome.

- Add an automated reconciliation test/check comparing ledger filled counts to relevant enrolment states.
- A counter must never be allowed to become negative or exceed capacity.

7. Enrolment lifecycle

State	Meaning in prototype	Seat effect (must be explicit)
STAGED	Row uploaded but not official.	No official seat counted.
PROVISIONAL	Validated/certified reservation state, as defined by approved workflow.	Reserve seat if the project rules say provisional enrolments occupy capacity.
CONFIRMED	Confirmed enrolment.	Counts as occupied under the configured rules.
WITHDRAWN	Student no longer occupies the admission.	Release seat once, with history.
TRANSFERRED	Enrolment moved to another institution/programme.	Release old scope only when the transition is accepted; track destination case.
COMPLETED	Programme completion.	Do not treat as current intake occupancy; retain historical record.
DROPPED	Enrolment ended for documented reason.	Release according to policy; keep history.

This is a proposed state contract aligned to the lifecycle concepts in the source report. Exact transitions and seat effects must be encoded in one transition table and tested; do not allow arbitrary direct status changes.

8. Conflict workflow

1. Detect candidate conflict from studentRef + academic year + enrolment state.
2. Create a Conflict record with both enrolment references and safe display fields.
3. Notify responsible institution users and authorised regional/national officers.
4. Allow institution to submit the supported resolution (withdrawn/transferred/justified) with a reason.
5. Run the state transition and seat release/reserve consistently; append history and audit.
6. Close only when the configured resolution conditions are satisfied; otherwise keep open/escalate.

9. Required tests

- Every validation layer has at least one passing and one failing test.
- Duplicate detection reports both relevant row numbers and does not merge silently.
- Configured bucket capacity 3 rejects a fourth reservation.
- Allowed additional buckets do not trigger an incorrect total-only overflow.
- Concurrent final-seat reservation yields at most one success.
- A failed transaction leaves neither official enrolment nor filled counter incremented.
- Seat release on withdrawal happens exactly once.
- Transfer creates/reconciles conflict and preserves historical state.
- Risk flag does not block or punish automatically.
- Policy-version mismatch forces revalidation or an explicit reject path.